

Configuración y Uso de Wazuh como SIEM

Correlación de eventos multi-fuente sobre un endpoint Debian con WordPress: agentes, logs de aplicación, integridad de archivos y autenticación

Gonzalo Rodriguez

4Geeks Academy — Ciberseguridad Blue Team / SOC

3 de julio de 2026

Índice

Para generar el índice automático: hacer clic aquí debajo y luego ir a la pestaña “Referencias” → “Tabla de contenido” → elegir un estilo automático. Word tomará los títulos numerados (Heading 1, 2 y 3) que ya están aplicados en todo el documento.

1. Introducción

El presente informe documenta la práctica de configuración y uso de Wazuh como SIEM (Security Information and Event Management), realizada como parte del curso de ciberseguridad Blue Team / SOC de 4Geeks Academy. El objetivo del ejercicio es demostrar cómo Wazuh, además de funcionar como un EDR sobre un endpoint individual, puede recopilar y correlacionar eventos provenientes de múltiples fuentes de datos dentro de un mismo sistema, generando alertas que permiten identificar patrones de ataque distribuidos.

Para esto se utilizó una máquina virtual Debian 12 con un sitio WordPress ya desplegado, sobre la cual se instaló un agente de Wazuh y se configuraron dos fuentes de datos: los eventos propios del agente (autenticación del sistema, integridad de archivos) y los logs generados por el servidor web Apache y por WordPress. Sobre esa base se simularon distintos escenarios de ataque y se analizaron los resultados desde el módulo Threat Hunting del dashboard de Wazuh.

2. Objetivos

Configurar Wazuh para recibir información desde múltiples fuentes de datos de un mismo endpoint, incluyendo el propio agente instalado y los logs de las aplicaciones que corren sobre él (Apache y WordPress).

Simular una serie de eventos de seguridad representativos de un ataque real: intentos de autenticación fallidos, modificación de un archivo sensible del sistema y generación de tráfico anómalo sobre el servidor web.

Verificar en el dashboard de Wazuh, específicamente en el módulo Threat Hunting, que los eventos generados quedan registrados y correlacionados correctamente.

Generar un reporte formal desde Wazuh que resuma los hallazgos del período analizado.

3. Entorno de trabajo

El laboratorio se desarrolló sobre el siguiente entorno virtualizado:

Rol	Sistema operativo	Dirección IP	Detalle
Wazuh Manager / Dashboard	Wazuh OVA v4.14.6	192.168.1.8	Instalación all-in-one (manager, indexer y dashboard en la misma VM)
Endpoint monitoreado	Debian GNU/Linux 12	192.168.1.10	Servidor Apache con sitio WordPress 6.9.4 desplegado en /var/www/html/wordpress

El agente Wazuh se registró en el manager bajo el nombre “Debian”, con ID de agente 003, grupo “default”. Cabe aclarar que, previo a esta práctica, ya se contaban con agentes de Wazuh instalados y funcionando en máquinas Kali y Linux Mint como parte de ejercicios anteriores del curso (EDR); el endpoint Debian con WordPress no tenía agente instalado, por lo que este fue el primer paso a resolver.

4. Desarrollo

4.1 Instalación del agente Wazuh en el endpoint Debian

El primer paso consistió en desplegar el agente de Wazuh sobre la máquina Debian con WordPress, dado que hasta ese momento no contaba con monitoreo. Para esto se utilizó el asistente “Deploy new agent” disponible en el propio dashboard de Wazuh (<https://192.168.1.8>), que genera de forma automática el comando de instalación correspondiente al sistema operativo seleccionado (Linux, distribución Debian, paquete .deb) e incluye la dirección del manager y el nombre asignado al agente.

Una vez ejecutado el comando en la terminal de la Debian, el agente quedó registrado y visible como activo en el dashboard, con la IP 192.168.1.10 correspondiente a esa máquina.

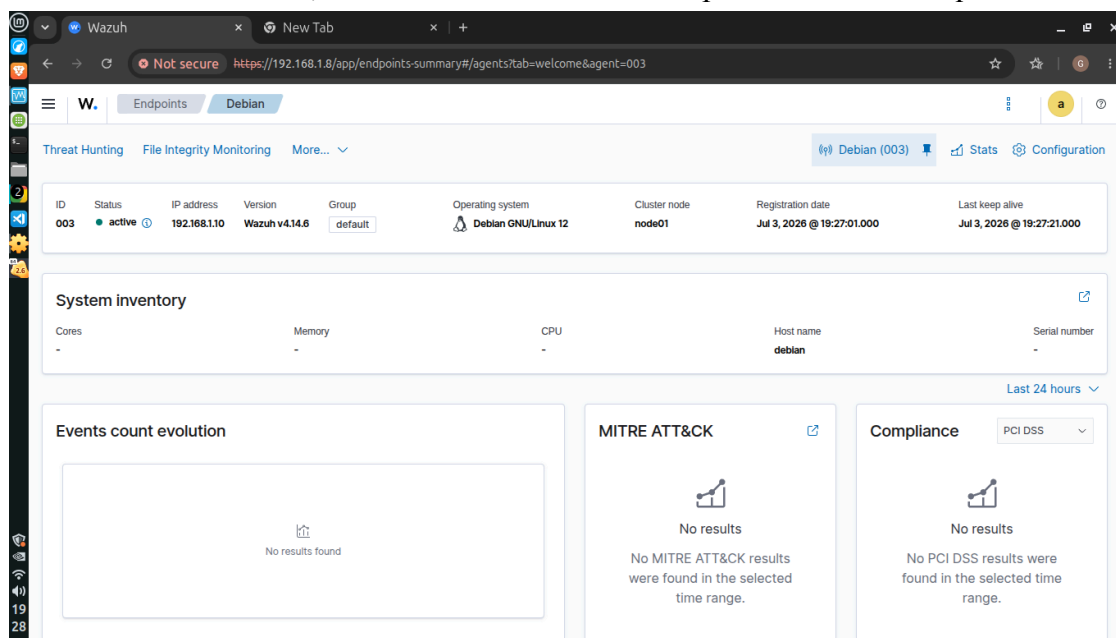


Figura 1. Agente Wazuh “Debian” (ID 003) registrado y activo en el dashboard, IP 192.168.1.10.

4.2 Configuración de fuentes de datos

Como se explica en la consigna del ejercicio, un SIEM recopila información desde diversas fuentes: agentes en los propios endpoints, logs de aplicaciones, y logs de dispositivos de red como firewalls o routers. En esta práctica se trabajó específicamente con las dos primeras fuentes.

4.2.1 Fuente 1: agente Wazuh en el endpoint

Esta fuente corresponde directamente al agente instalado en el paso anterior, el cual reporta al manager información de autenticación del sistema (a través de PAM y su registro de accesos), uso de sudo, y el estado de integridad de archivos mediante el módulo syscheck, ya habilitado por defecto sobre directorios sensibles como /etc, /usr/bin, /usr/sbin, /bin, /sbin y /boot.

4.2.2 Fuente 2: logs de aplicaciones (Apache y WordPress)

Para esta fuente se revisó el archivo de configuración del agente (/var/ossec/etc/ossec.conf), donde se constató que ya existían los bloques <localfile> necesarios para que Wazuh recolectara los logs de acceso y error de Apache (/var/log/apache2/access.log y /var/log/apache2/error.log), además de bloques adicionales para journald y para el log de instalación de paquetes (dpkg.log). Esto simplificó el trabajo, ya que no fue necesario agregar manualmente dichas entradas.

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/error.log</location>
</localfile>

<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/access.log</location>
</localfile>
```

Se verificó que los archivos de log de Apache existían en el sistema de archivos y que el servicio apache2 se encontraba activo y en ejecución.

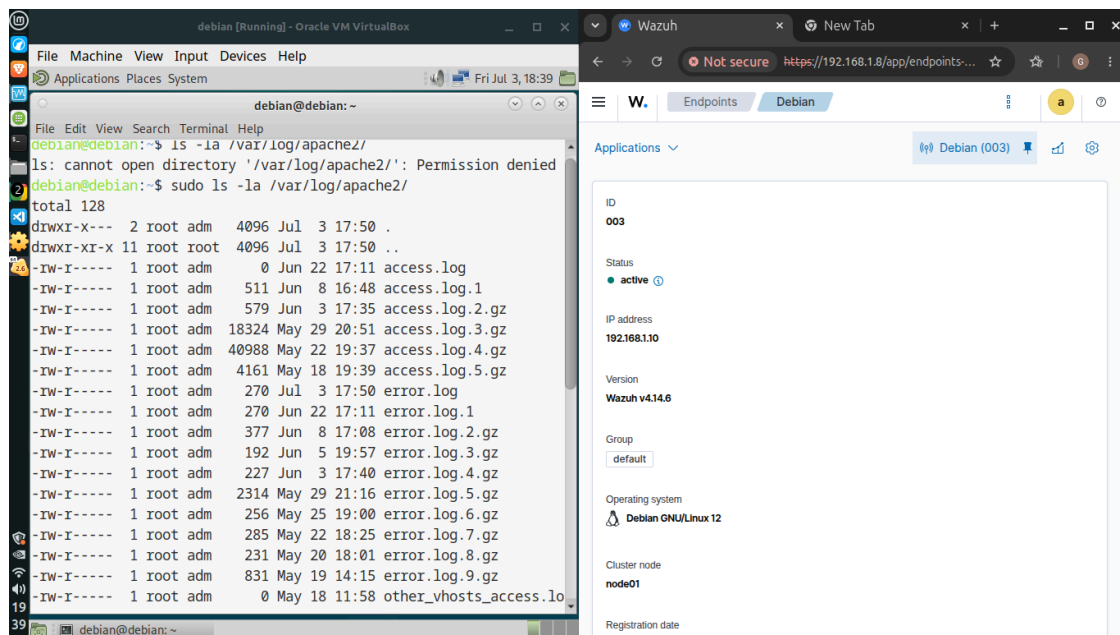


Figura 2. Listado de los archivos de log de Apache en /var/log/apache2/, confirmando la existencia de access.log y error.log.

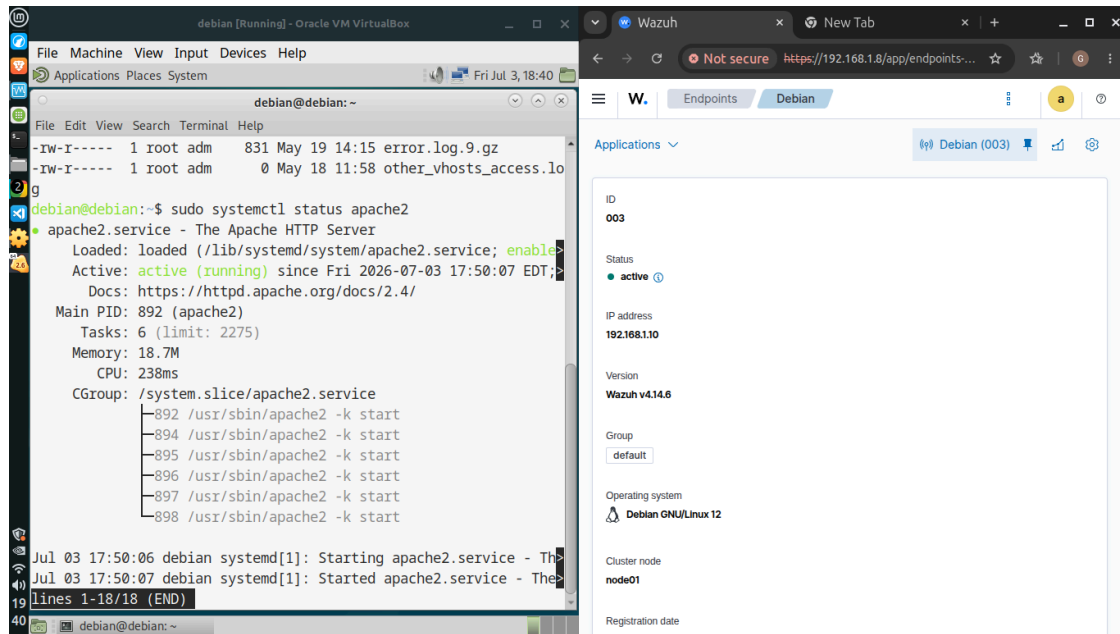


Figura 3. Estado del servicio `apache2`, activo (*running*).

Adicionalmente, dado que WordPress no genera logs propios de forma predeterminada, se habilitó el modo de depuración editando el archivo `wp-config.php` ubicado en `/var/www/html/wordpress/`, reemplazando el valor original de `WP_DEBUG` (que se encontraba en `false`) e incorporando las directivas necesarias para registrar errores y advertencias de PHP en un archivo de log dedicado, sin mostrarlos en pantalla.

```
define( 'WP_DEBUG', true );
define( 'WP_DEBUG_LOG', true );
define( 'WP_DEBUG_DISPLAY', false );
```

Se realizó previamente una copia de resguardo del archivo original (`wp-config.php.bak`) antes de aplicar el cambio, como buena práctica.

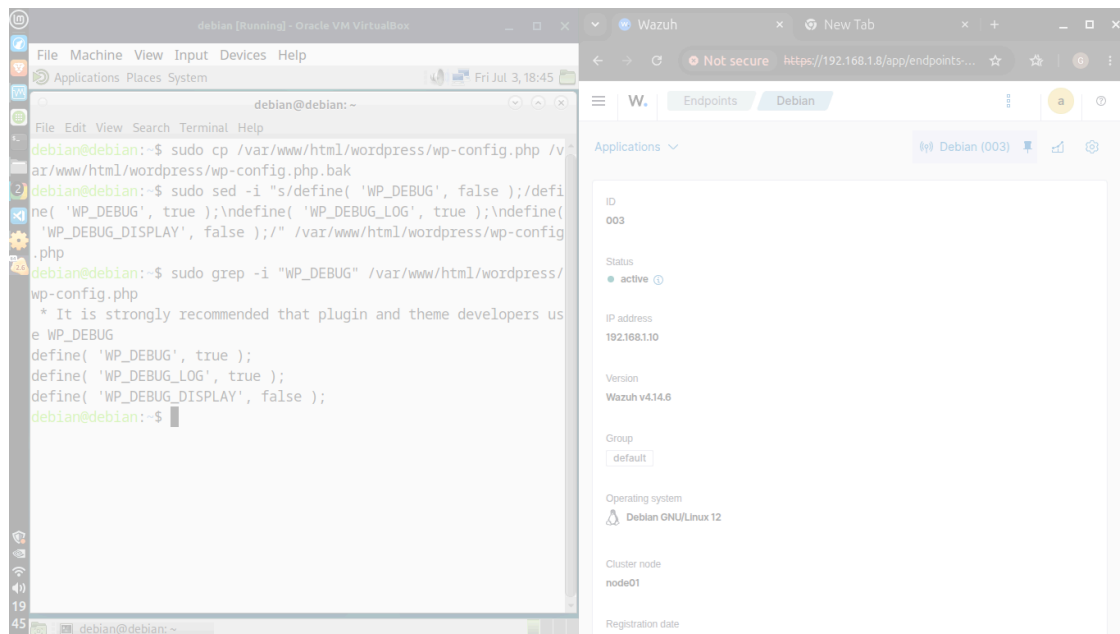


Figura 4. Verificación de las tres directivas `WP_DEBUG` configuradas correctamente en `wp-config.php`. Finalmente, se reinició el servicio del agente Wazuh para asegurar que todos los módulos (`agentd`, `syscheckd`, `logcollector`, `modulesd`) tomaran la configuración vigente y comenzaran a recolectar la información desde cero.

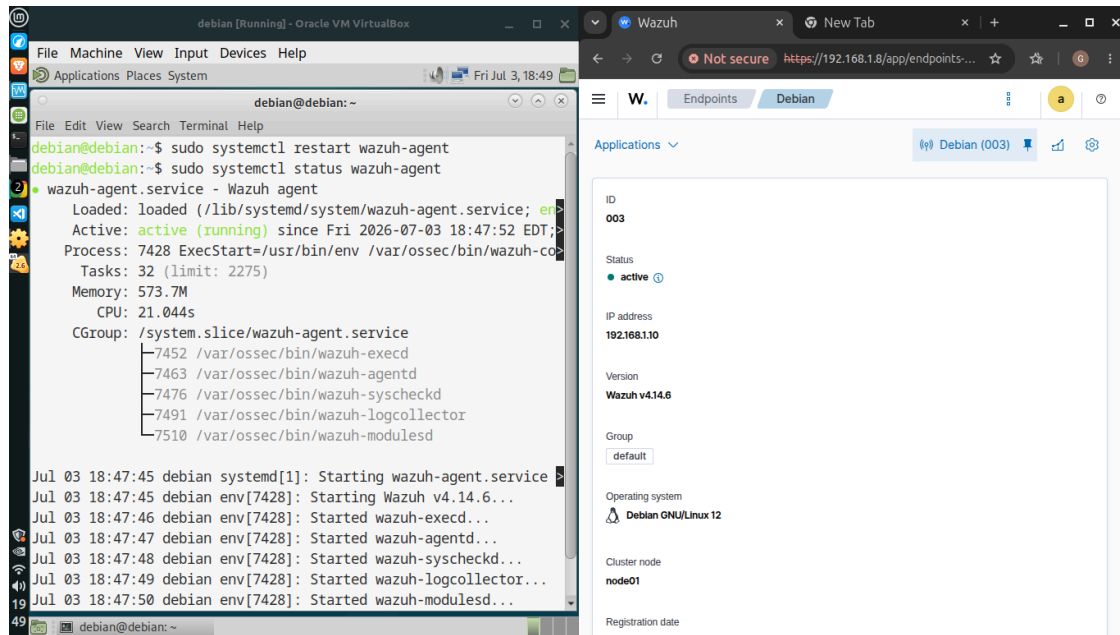


Figura 5. Reinicio del agente Wazuh; todos los módulos internos inician correctamente.

4.3 Simulación de ataques multi-fuente

Con las fuentes de datos ya configuradas, se procedió a simular tres tipos de eventos de seguridad sobre el endpoint Debian, buscando generar actividad detectable desde distintos orígenes y así evaluar la capacidad de correlación de Wazuh.

4.3.1 Intentos de autenticación fallidos

Se generaron múltiples intentos de acceso con credenciales incorrectas utilizando el comando `su`, repitiendo el intento con contraseñas inválidas en once oportunidades consecutivas. Este tipo de evento es representativo de un intento de acceso no autorizado o ataque de fuerza bruta, y se corresponde con la técnica T1078 de uso indebido de credenciales del framework MITRE ATT&CK.

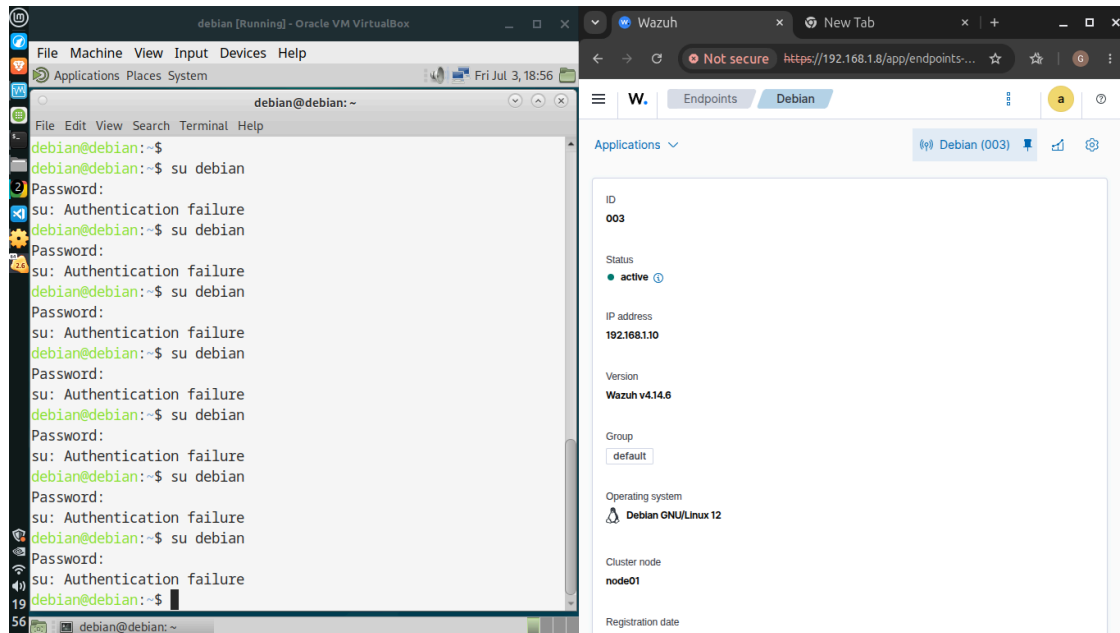


Figura 6. Once intentos de autenticación fallidos (`su: Authentication failure`) registrados en la terminal del endpoint.

4.3.2 Modificación de un archivo sensible

Se simuló la alteración de un archivo crítico del sistema, agregando una línea de texto arbitraria al final de `/etc/passwd` mediante el comando `sudo su -c 'echo "Simulacion de cambio malicioso" >> /etc/passwd'`. Este archivo se encuentra bajo el alcance del módulo de integridad de archivos (syscheck) de Wazuh, por lo que cualquier alteración de su contenido debería ser detectada en el próximo ciclo de escaneo.

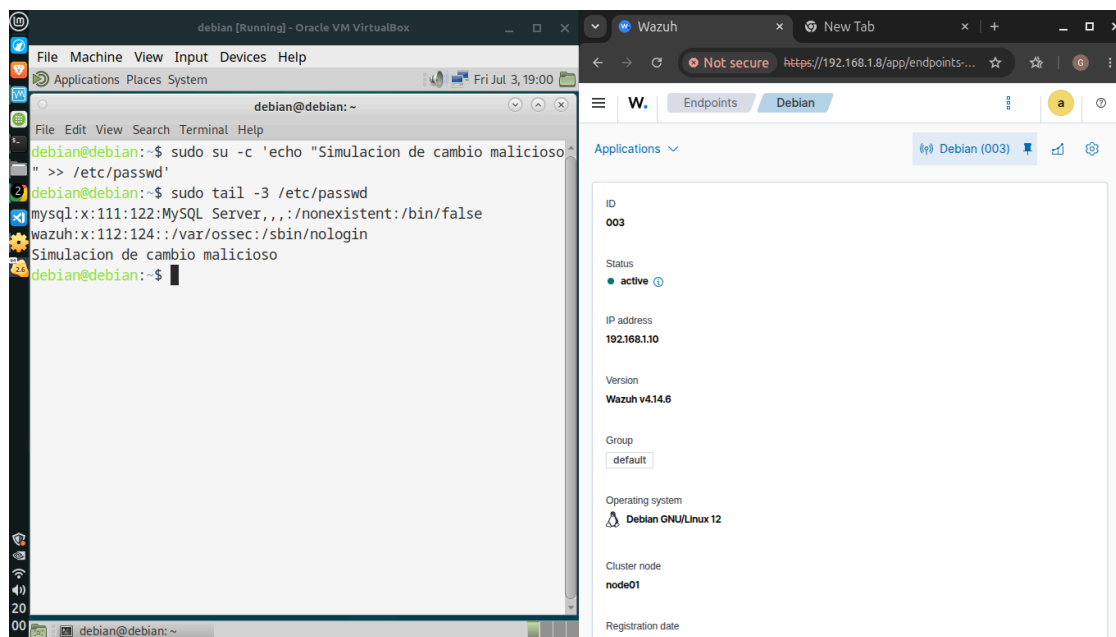


Figura 7. Confirmación, mediante tail, de la línea agregada al final de `/etc/passwd`.

Es importante señalar que la modificación fue verificada directamente a nivel de sistema de archivos; la correlación específica del cambio dentro del dashboard bajo el grupo de reglas syscheck no llegó a confirmarse de forma concluyente durante la sesión de trabajo, dado que dicho módulo ejecuta sus comparaciones según una frecuencia programada (por defecto cada doce horas) y al inicio del agente. Se forzó un reinicio del servicio wazuh-agent mediante `wazuh-control restart` para disparar un nuevo ciclo de escaneo, y se identificaron en el dashboard eventos asociados al uso de sudo sobre el archivo (reglas 5402 y 5403, y verificaciones del benchmark CIS 19007-19008), aunque no una alerta explícita de tipo “Integrity checksum changed” para `/etc/passwd` en el rango de tiempo consultado.

4.3.3 Tráfico anómalo sobre el servidor web

Se generó actividad de navegación normal sobre el sitio WordPress (accesos a la página principal, ejecución de wp-cron) y, a continuación, se forzaron múltiples solicitudes a rutas inexistentes del sitio para provocar respuestas HTTP 404, simulando un intento de reconocimiento o escaneo de recursos sobre el servidor.

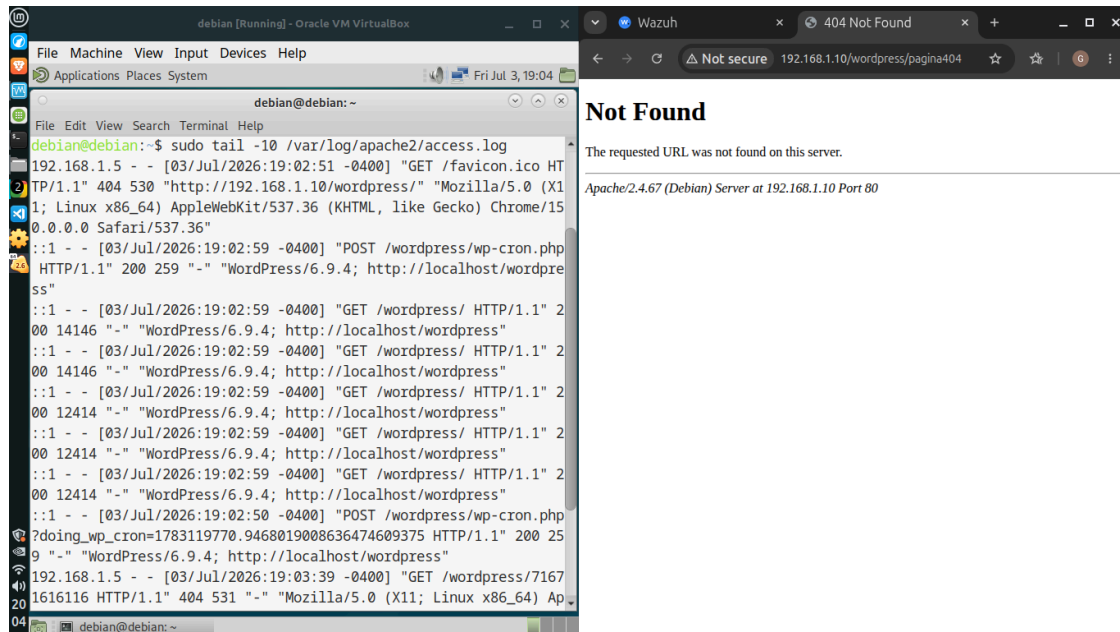


Figura 8. Registro en access.log: tráfico normal (200) junto con accesos a rutas inexistentes que generan código 404.

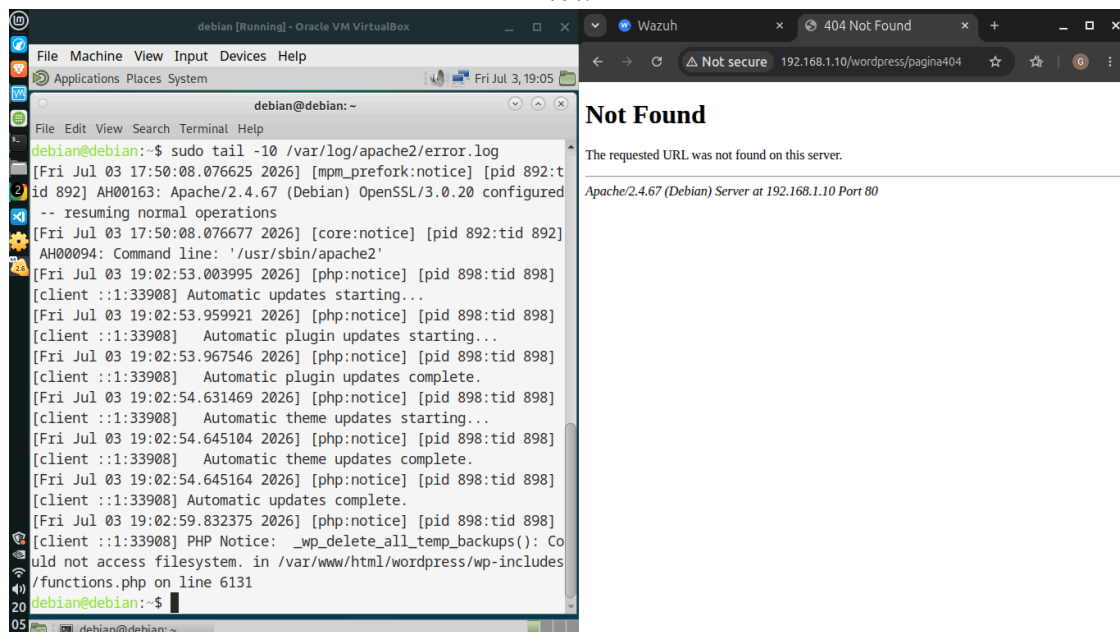


Figura 9. Registro en error.log, incluyendo un aviso (PHP Notice) generado por WordPress tras la activación de WP_DEBUG.

4.4 Análisis de eventos en Threat Hunting

Una vez generados los tres tipos de eventos, se accedió al módulo Threat Hunting del dashboard de Wazuh, filtrado por el agente Debian (ID 003), para revisar la correlación de la actividad registrada durante la ventana de tiempo del ejercicio.

La vista general mostró un total de 285 alertas en el período analizado, de las cuales 22 correspondieron a fallos de autenticación (authentication_failed) y 17 a autenticaciones exitosas (authentication_success), coincidiendo con los intentos fallidos generados mediante su y con las aperturas de sesión legítimas del propio usuario durante la práctica. Entre los grupos de alerta más frecuentes se destacaron pam, sudo, su, sca y syslog.

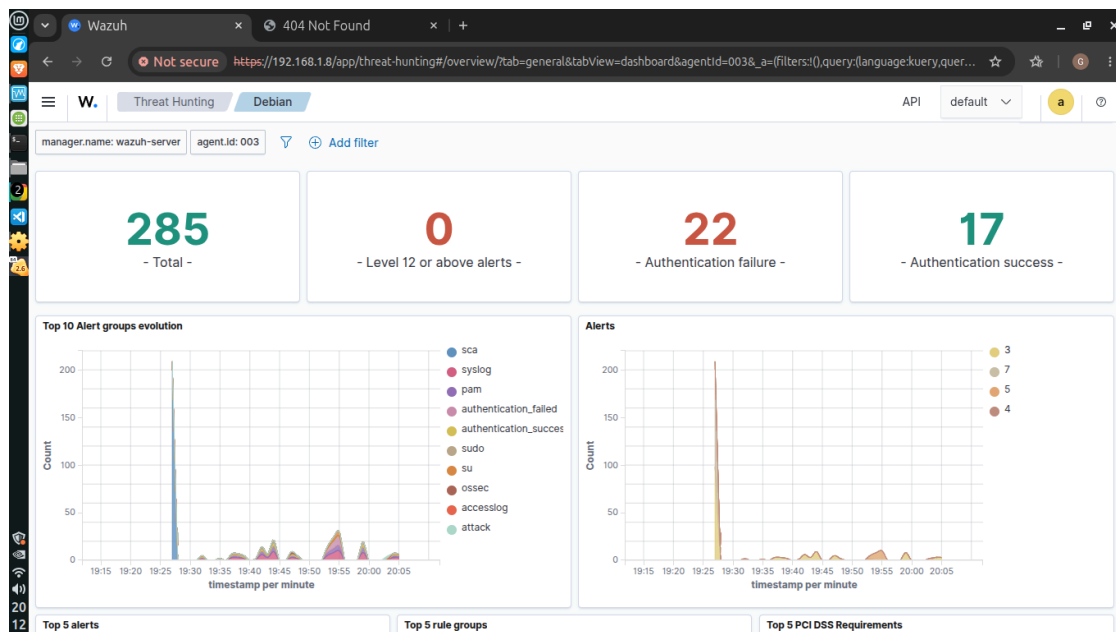


Figura 10. Vista general de Threat Hunting: 285 alertas totales, 22 fallos y 17 éxitos de autenticación.

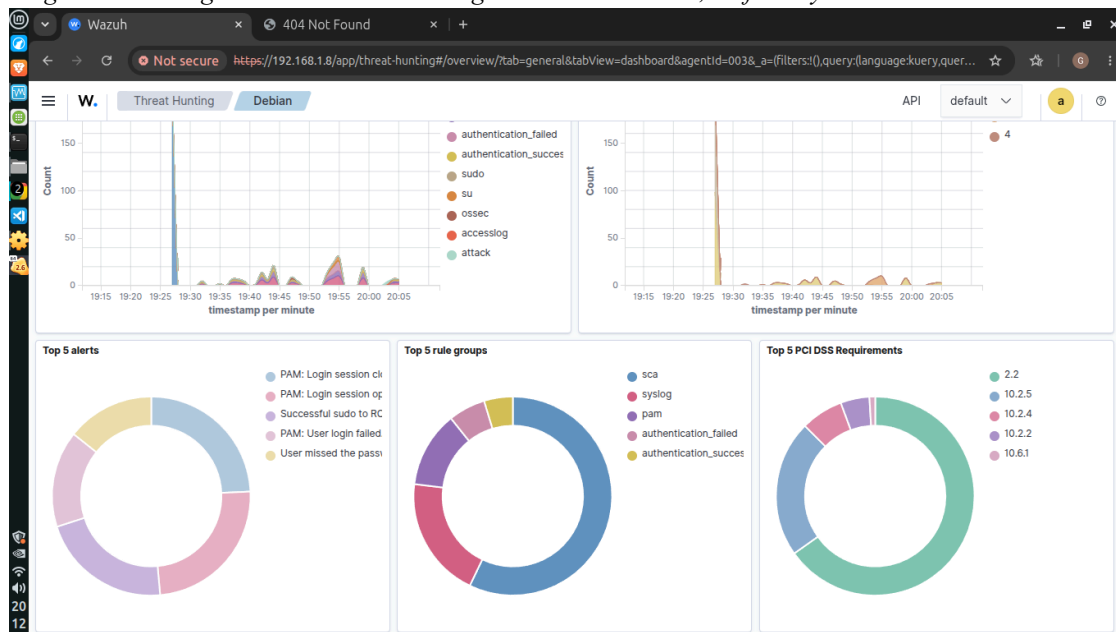


Figura 11. Top 5 de alertas y grupos de reglas, dominado por eventos PAM y de sudo.

Al filtrar los eventos por el término “404” se identificaron dos coincidencias asociadas a los grupos accesslog, attack y web, correspondientes a los accesos a rutas inexistentes generados sobre el sitio WordPress.

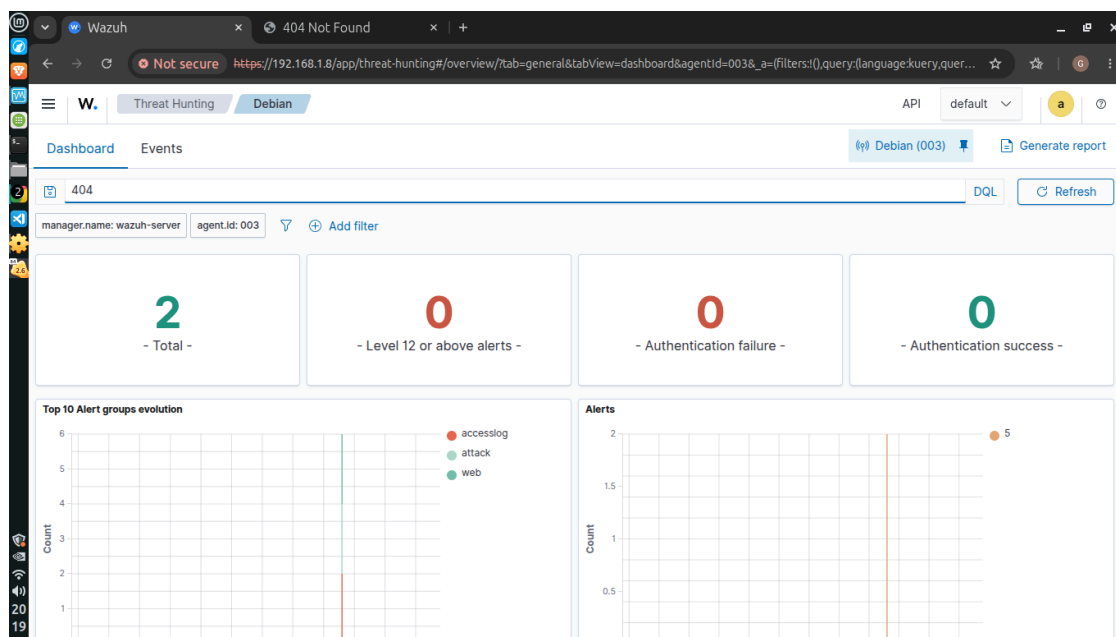


Figura 12. Resultado de la búsqueda “404”: dos eventos bajo los grupos accesslog, attack y web.

Al filtrar por el término “/etc/passwd” se obtuvieron siete coincidencias, correspondientes a eventos de uso de sudo sobre el sistema (reglas 5402 y 5403) y a verificaciones del benchmark de seguridad CIS para Debian 12 relacionadas con las cuentas del archivo /etc/passwd (reglas 19007 y 19008). Estos eventos confirman que el archivo fue objeto de operaciones auditadas por Wazuh, aunque, como se mencionó, no se logró visualizar en el dashboard una alerta específica de integridad de archivos para el cambio de contenido realizado.

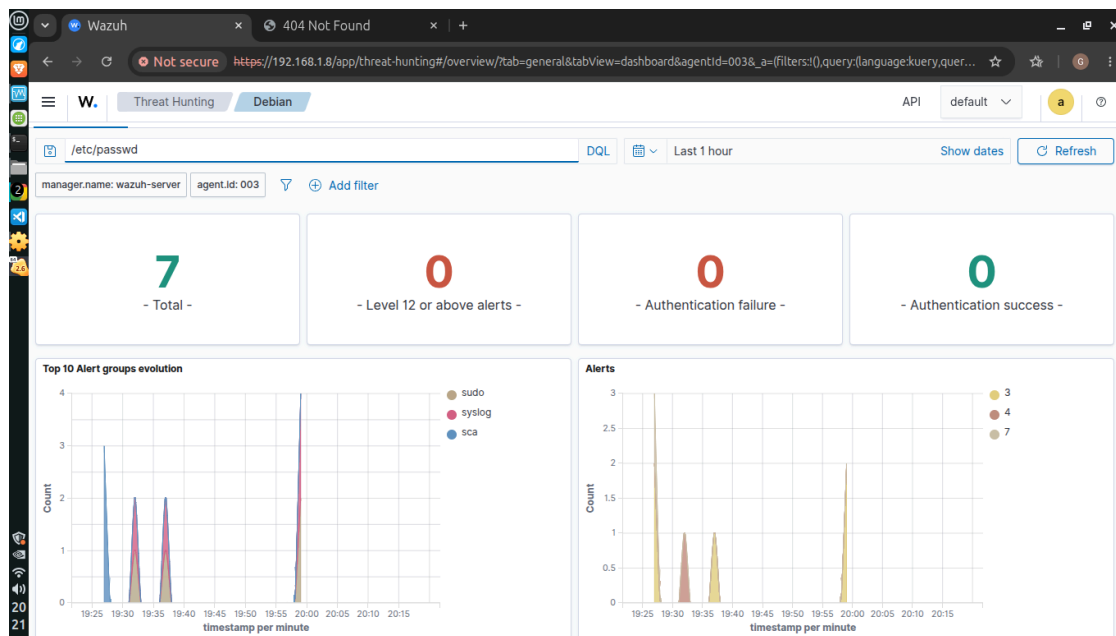


Figura 13. Resultado de la búsqueda “/etc/passwd”: siete eventos asociados a sudo y al benchmark CIS.

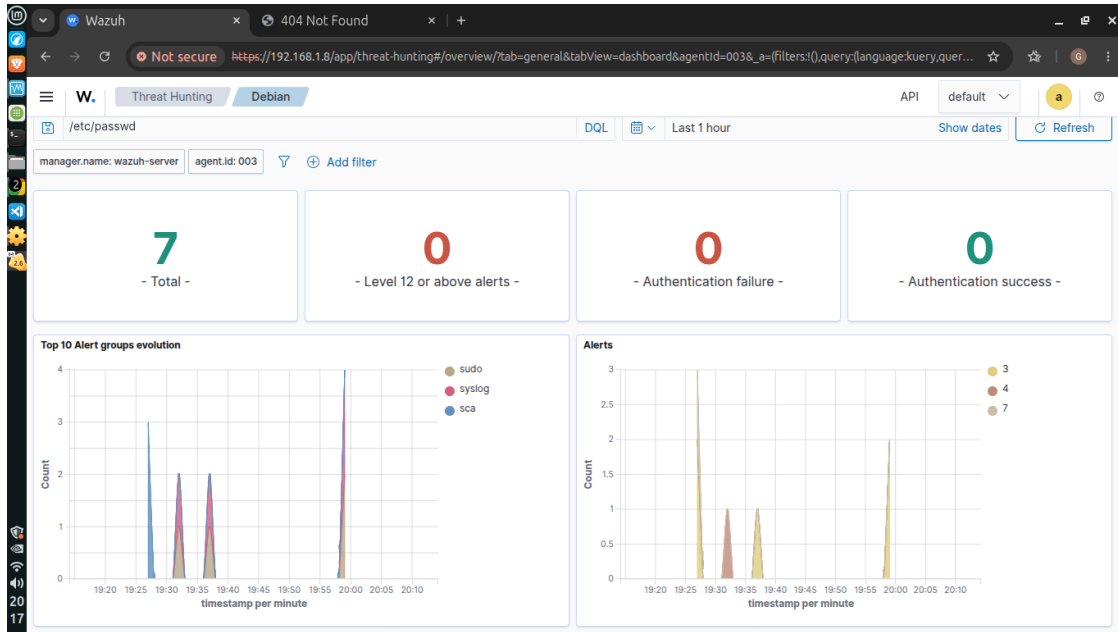


Figura 14. Detalle de la evolución temporal de los eventos relacionados con /etc/passwd (grupos sudo, syslog y sca).

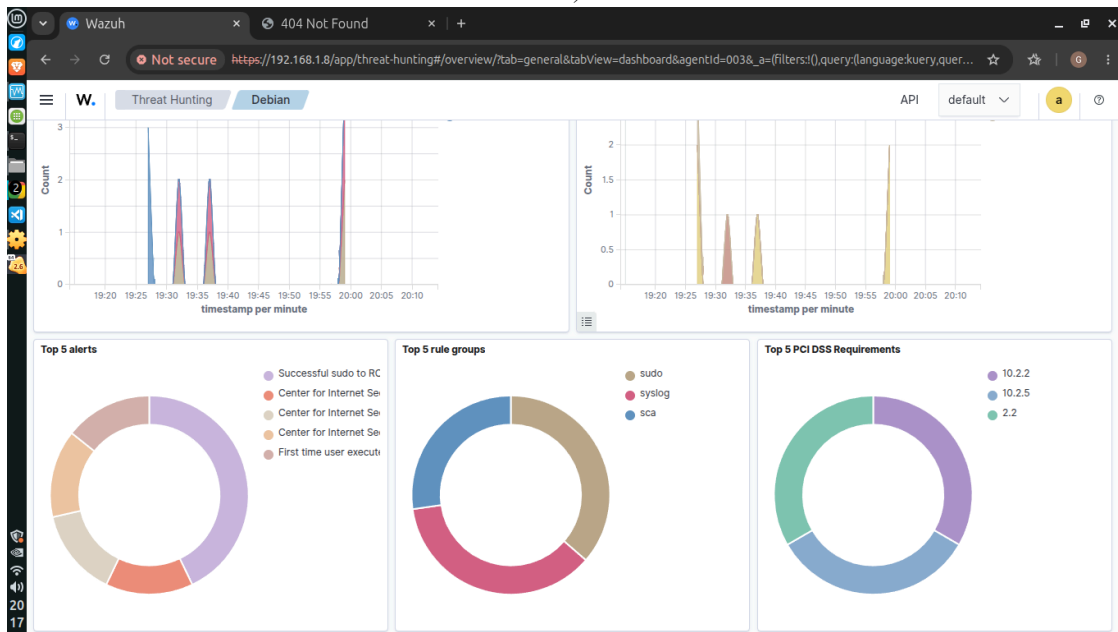


Figura 15. Top 5 de alertas y grupos de reglas para la búsqueda de /etc/passwd.

Finalmente, se revisó la tabla de eventos individuales (pestaña Events) filtrada por /etc/passwd, donde se detalla la descripción exacta de cada regla disparada: “Successful sudo to ROOT executed”, “First time user executed sudo”, y tres verificaciones del CIS Benchmark para Debian 12 Linux referidas a las cuentas del archivo de contraseñas del sistema.

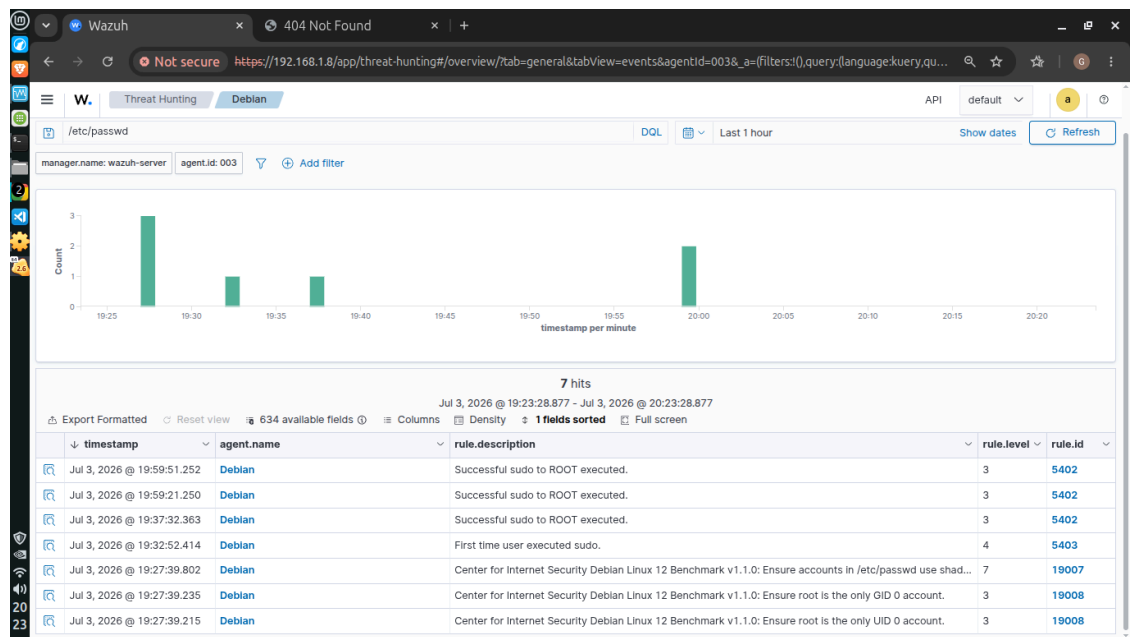


Figura 16. Tabla de eventos individuales filtrados por /etc/passwd, con su descripción, nivel y rule.id.

4.5 Generación del reporte en Wazuh

Como cierre del ejercicio, se generó desde el propio dashboard de Wazuh un reporte en formato PDF (Threat hunting report) correspondiente al agente Debian (ID 003), acotado a la ventana horaria en la que se realizaron las simulaciones. El reporte incluye la identificación del agente, la evolución temporal de los diez principales grupos de alertas, los contadores de autenticación, los cinco principales tipos de alerta y grupos de reglas, los requerimientos de PCI DSS asociados, y dos tablas de resumen: alertas por regla y alertas por grupo.

Según el resumen final del reporte, se registraron 81 alertas en la ventana consultada, con 22 fallos de autenticación y 18 autenticaciones exitosas. Las reglas más frecuentes correspondieron a apertura y cierre de sesión PAM (18 ocurrencias cada una), ejecución exitosa de sudo hacia root (16 ocurrencias), fallos de contraseña en cambios de UID y fallos de login PAM (11 ocurrencias cada una), y dos ocurrencias de código de error 400 del servidor web. A nivel de grupos, syslog (75) y pam (47) concentraron la mayor parte de la actividad, seguidos por authentication_failed (22), authentication_success (18), sudo (17) y su (11); los grupos accesslog, attack y web, vinculados al tráfico anómalo de Apache, registraron dos eventos cada uno.

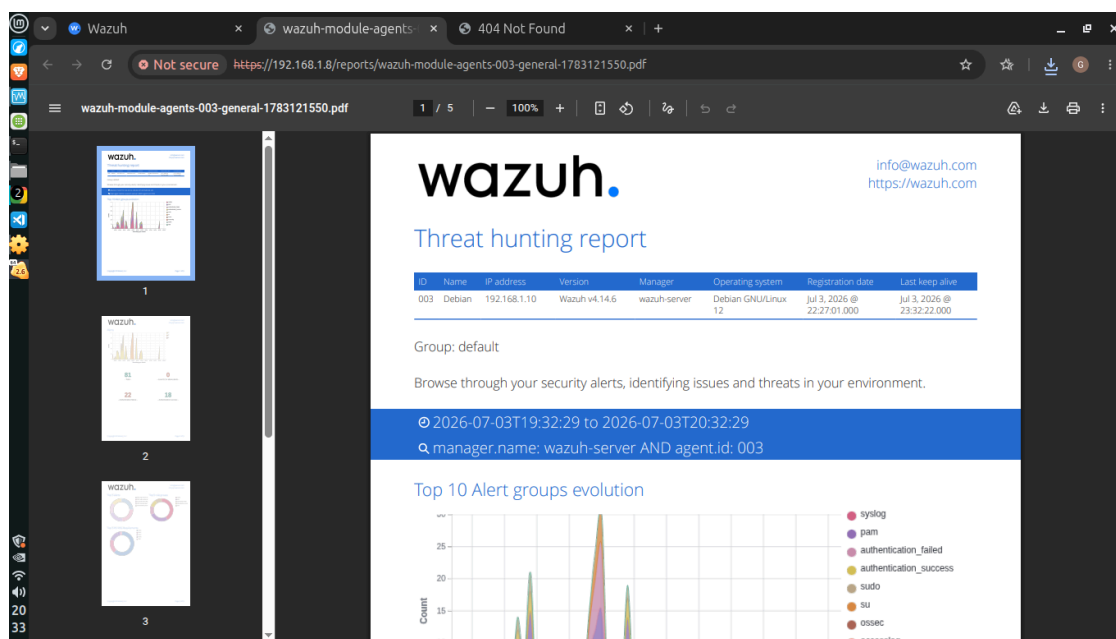


Figura 17. Reporte PDF generado desde Wazuh (wazuh-module-agents-003-general), primera página con la identificación del agente.

Esta tabla resume los principales hallazgos consolidados en el reporte final:

Regla (rule.id)	Descripción	Nivel	Ocurrencias
5501	PAM: Login session opened	3	18
5502	PAM: Login session closed	3	18
5402	Successful sudo to ROOT executed	3	16
5301	User missed the password to change UID (user id)	5	11
5503	PAM: User login failed	5	11
31101	Web server 400 error code	5	2
503	Wazuh agent started	3	2
506	Wazuh agent stopped	3	2
5403	First time user executed sudo	4	1

5. Resultados

La práctica permitió comprobar de manera práctica el rol de Wazuh como SIEM: un mismo endpoint, correctamente configurado, es capaz de aportar evidencia proveniente de fuentes heterogéneas (autenticación de sistema, uso de privilegios, tráfico de aplicación web) que el manager consolida y correlaciona en una única vista de análisis.

De las tres simulaciones realizadas, dos quedaron claramente confirmadas y correlacionadas en el dashboard: los intentos de autenticación fallidos, reflejados en los contadores de authentication_failed y en los grupos pam y su, y el tráfico anómalo de Apache, reflejado en los grupos accesslog, attack y web asociados al código 404. La tercera simulación, la modificación de /etc/passwd, quedó confirmada a nivel de sistema de archivos y generó eventos indirectos relacionados (uso de sudo, verificaciones CIS), pero no se logró visualizar una alerta explícita del módulo syscheck durante la ventana de tiempo trabajada, lo cual se atribuye a la frecuencia de escaneo programada de dicho módulo.

6. Conclusiones

El ejercicio permitió reforzar la comprensión del funcionamiento de Wazuh no solo como una herramienta de protección de endpoints (EDR), sino como una plataforma de correlación de eventos de seguridad a nivel SIEM, capaz de integrar información de múltiples orígenes dentro de un mismo sistema y presentarla de forma unificada para su análisis.

Asimismo, quedó en evidencia la importancia de comprender los tiempos y ciclos de ejecución de cada módulo de Wazuh, particularmente en el caso de syscheck, cuya frecuencia de escaneo programada puede generar una demora entre el momento en que ocurre un cambio real en el sistema y el momento en que dicho cambio se refleja como alerta en el dashboard. Esto resulta relevante desde una perspectiva operativa, ya que en un entorno productivo dicha latencia debería ajustarse según el nivel de criticidad de los archivos monitoreados.

Como mejora a futuro, se identificó la necesidad de asignar una dirección IP estática al manager de Wazuh, dado que actualmente opera bajo DHCP y un cambio de dirección puede provocar la pérdida de comunicación con los agentes ya registrados.